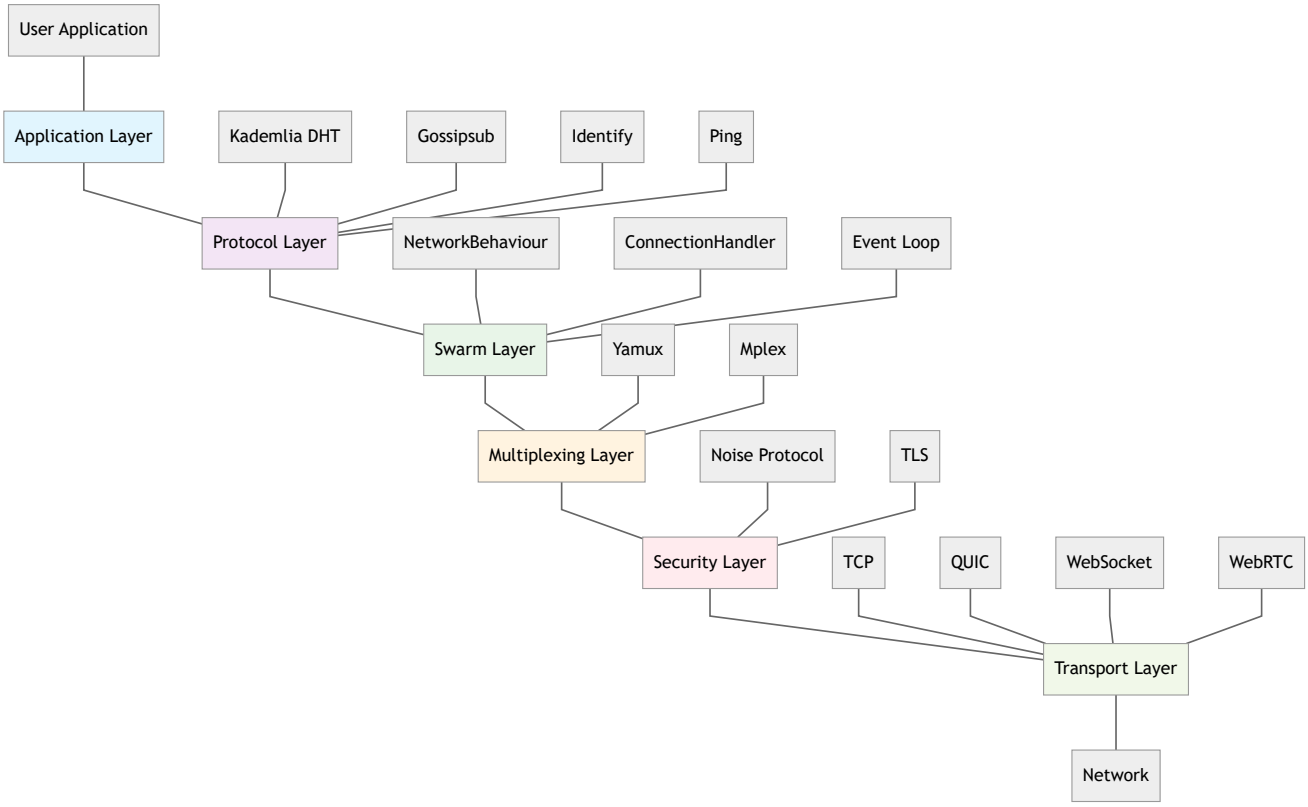
Rust libp2p 데이터 흐름 다이어그램

개요

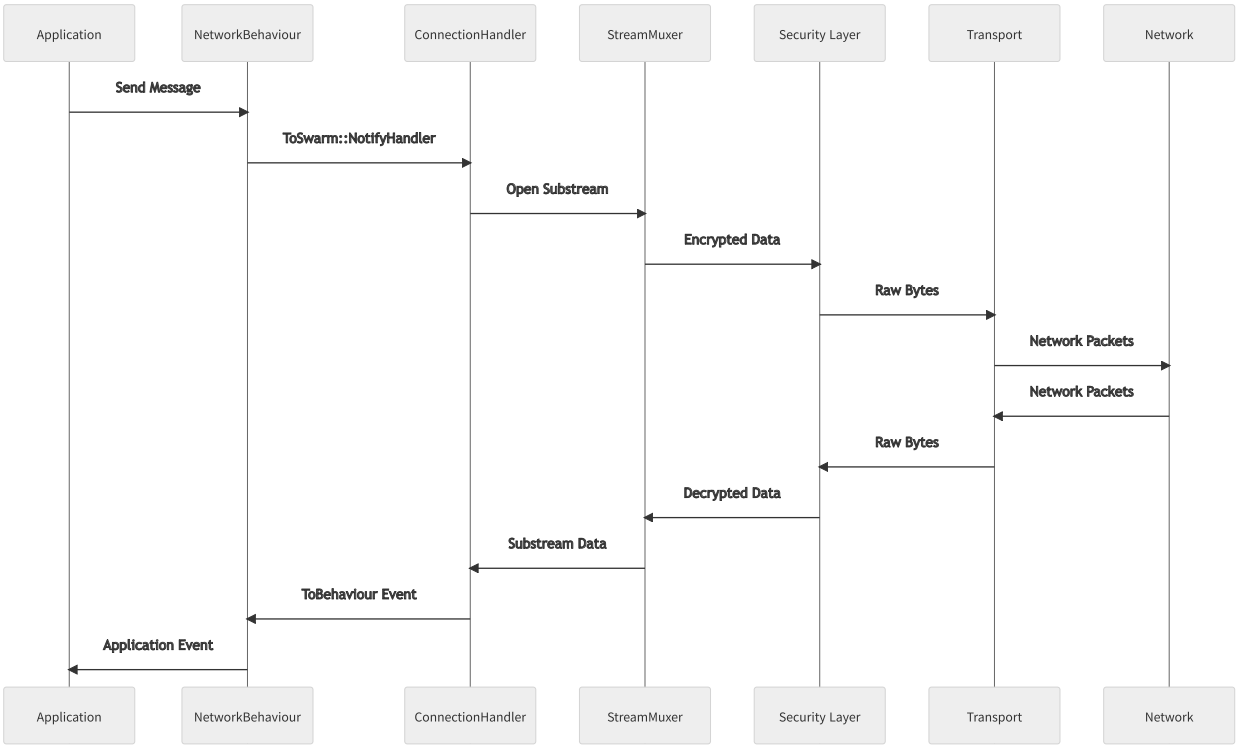
이 문서는 Rust libp2p 구현체의 데이터 흐름을 시각적으로 표현한 다이어그램들을 제공합니다. 각 계층별 데이터 흐름과 구성 요소 간의 상호작용을 명확히 보여줍니다.

1. 전체 아키텍처 데이터 흐름

1.1 계층별 데이터 흐름 개요

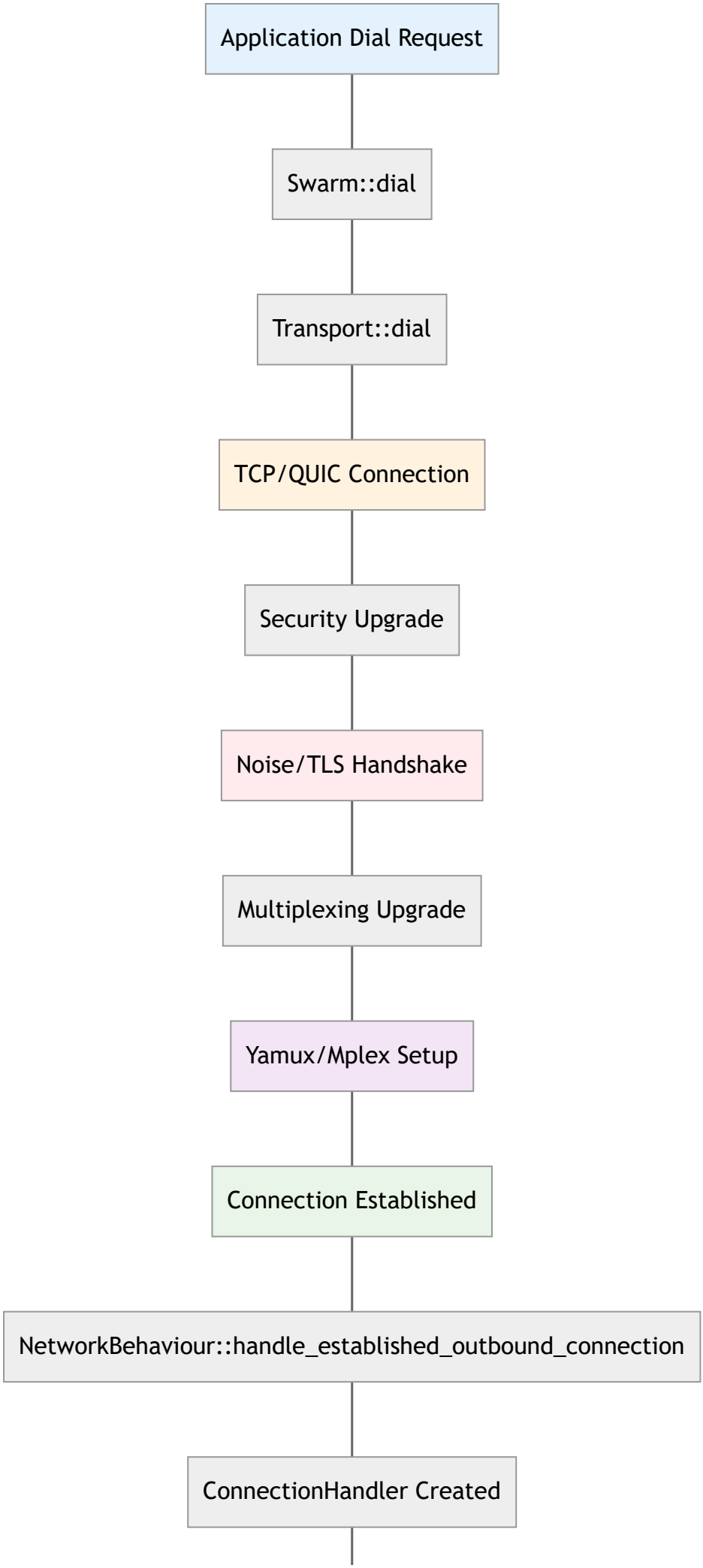


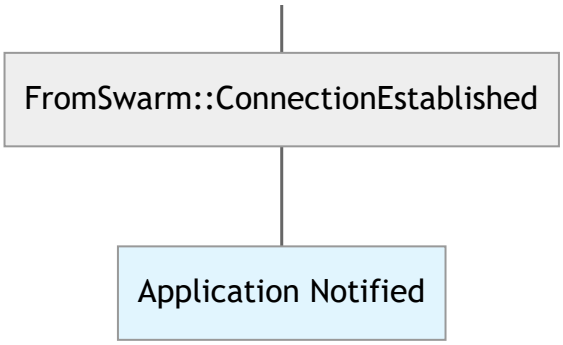
1.2 메시지 전송 데이터 흐름



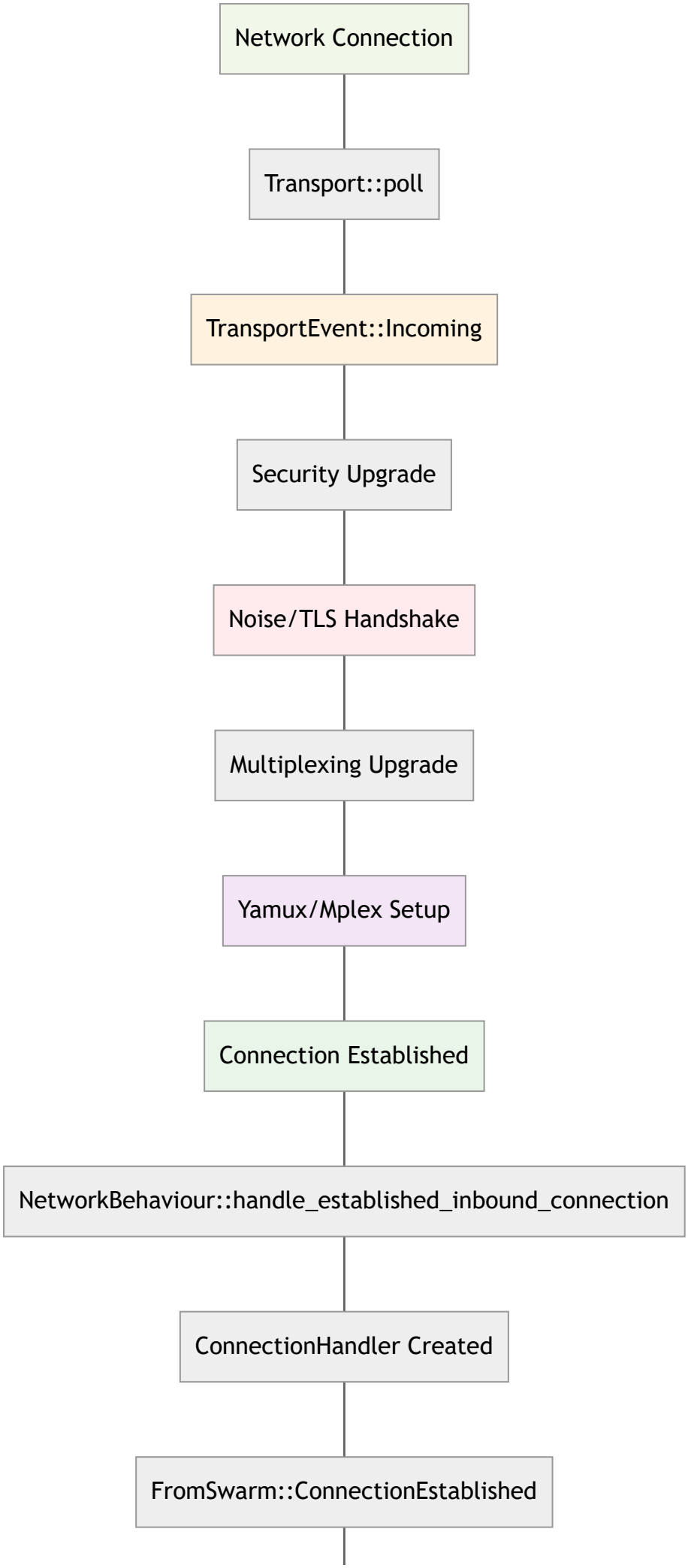
2. 연결 설정 데이터 흐름

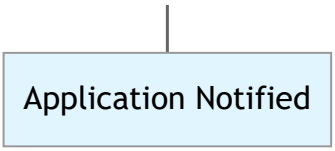
2.1 아웃바운드 연결 설정


Application Dial Request
Swarm::dial
Transport::dial
TCP/QUIC Connection
Security Upgrade
Noise/TLS Handshake
Multiplexing Upgrade
Yamux/Mplex Setup
Connection Established
NetworkBehaviour::handle_established_outbound_connection
ConnectionHandler Created



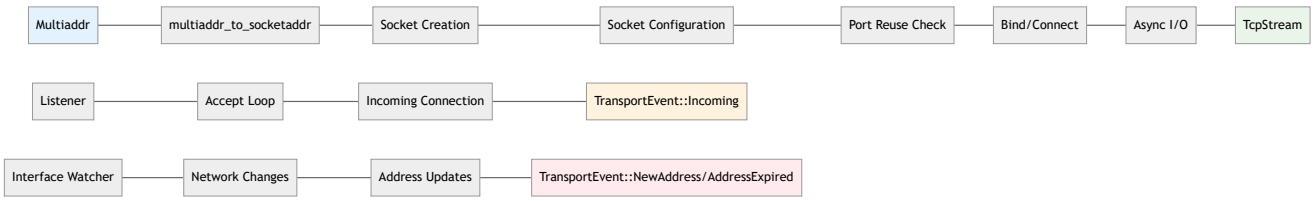
2.2 인바운드 연결 설정


Network Connection
Transport::poll
TransportEvent::Incoming
Security Upgrade
Noise/TLS Handshake
Multiplexing Upgrade
Yamux/Mplex Setup
Connection Established
NetworkBehaviour::handle_established_inbound_connection
ConnectionHandler Created
FromSwarm::ConnectionEstablished

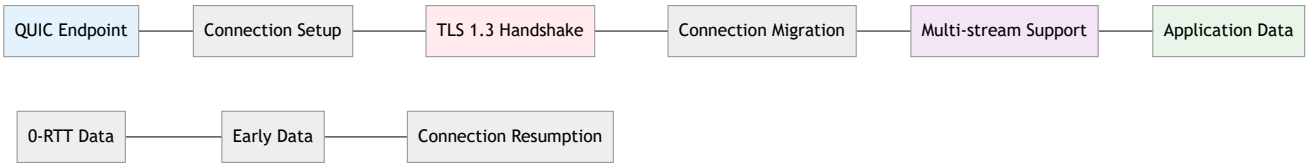


3. Transport Layer 데이터 흐름

3.1 TCP Transport 데이터 흐름

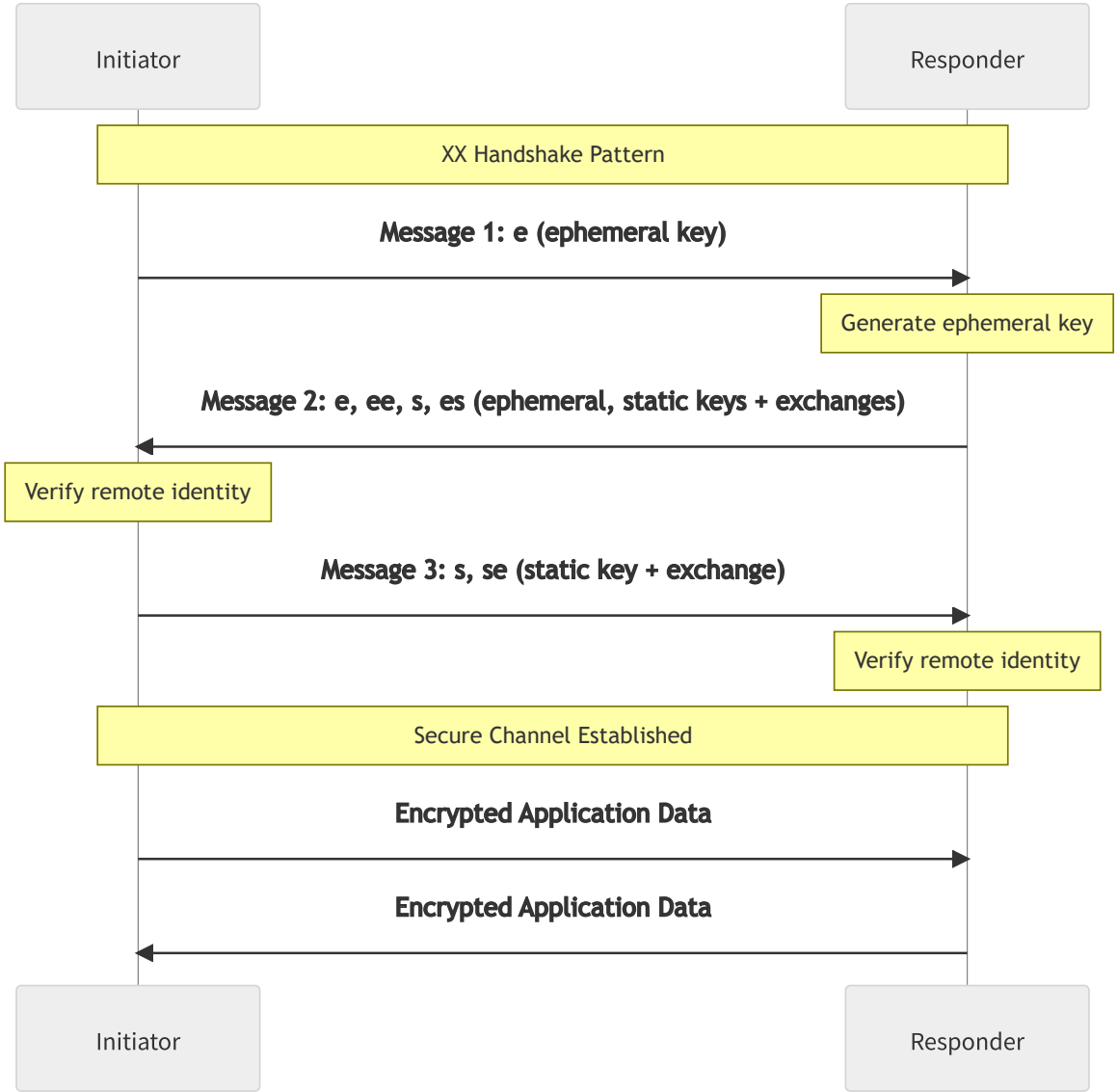


3.2 QUIC Transport 데이터 흐름

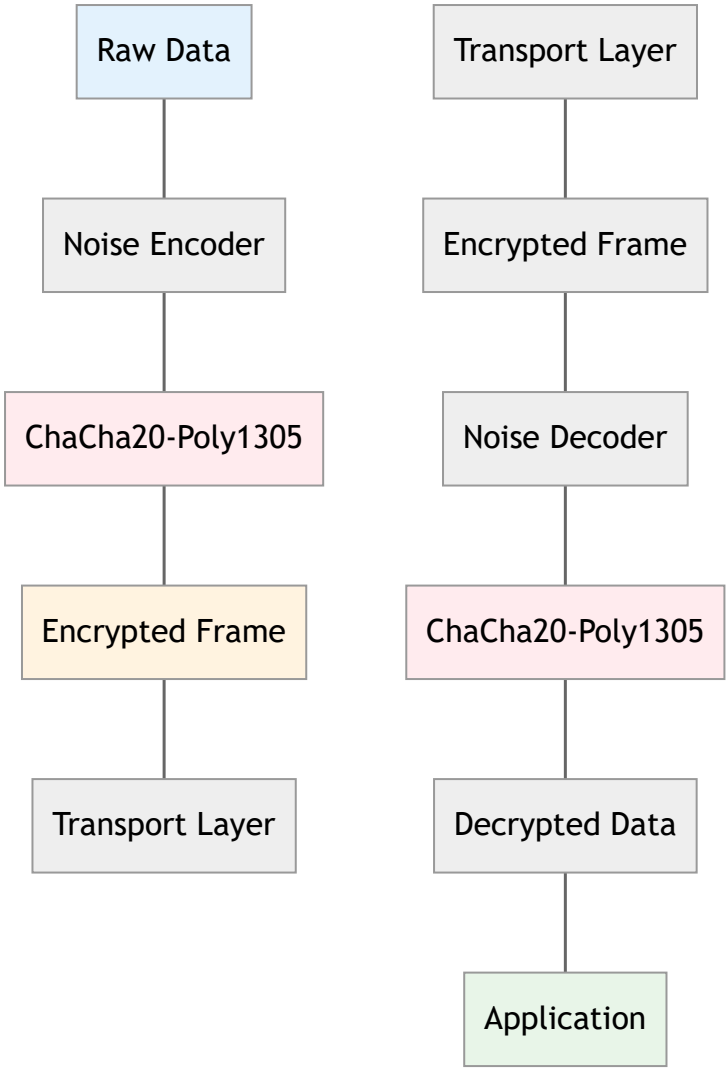


4. Security Layer 데이터 흐름

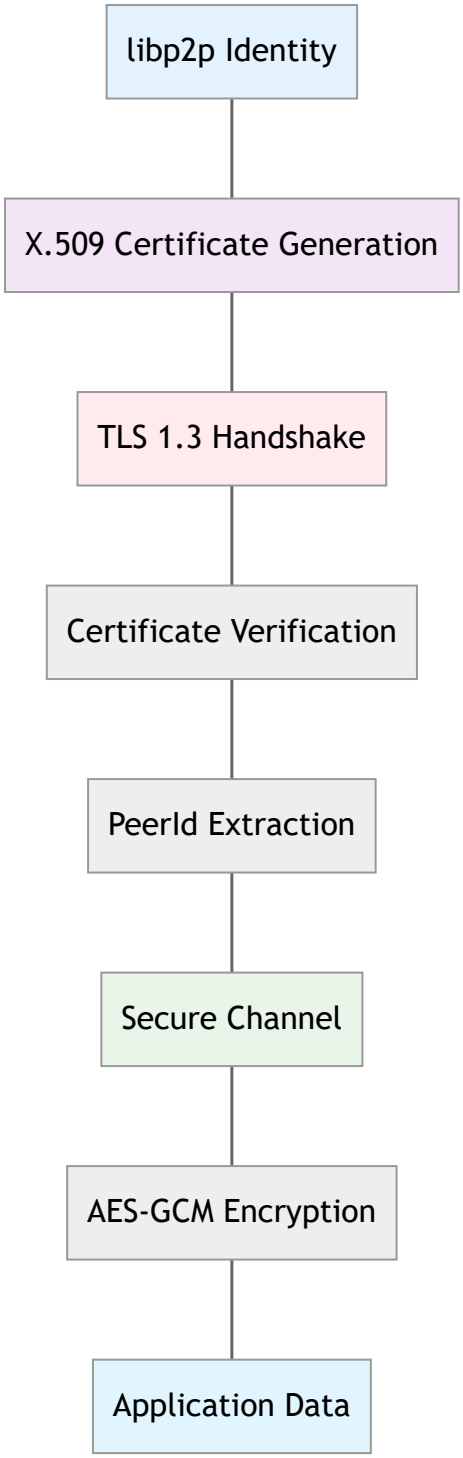
4.1 Noise Protocol 핸드셰이크



4.2 Noise Protocol 데이터 처리

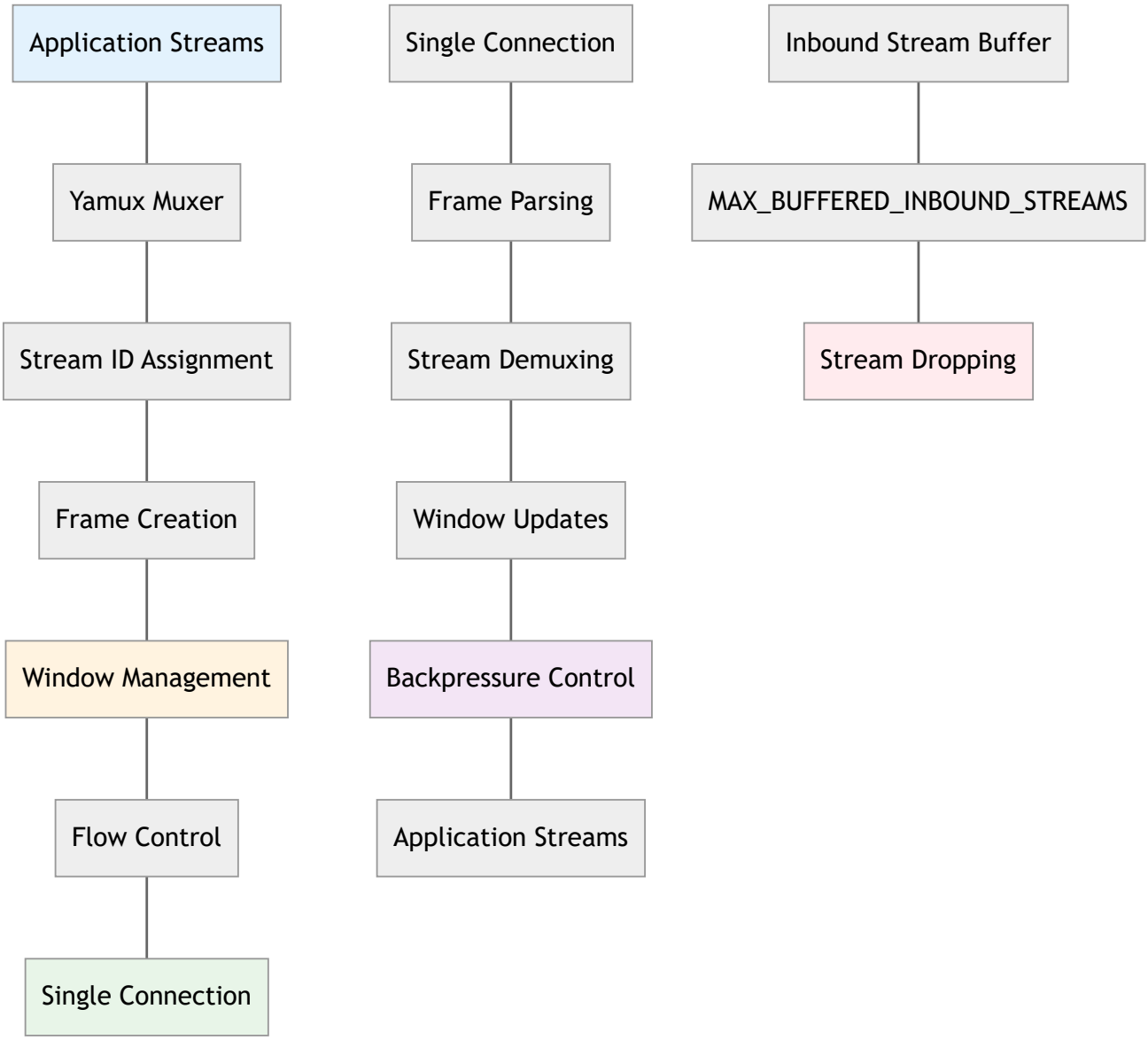

Raw Data
Noise Encoder
ChaCha20-Poly1305
Encrypted Frame
Transport Layer
Transport Layer
Encrypted Frame
Noise Decoder
ChaCha20-Poly1305
Decrypted Data
Application

4.3 TLS 데이터 흐름

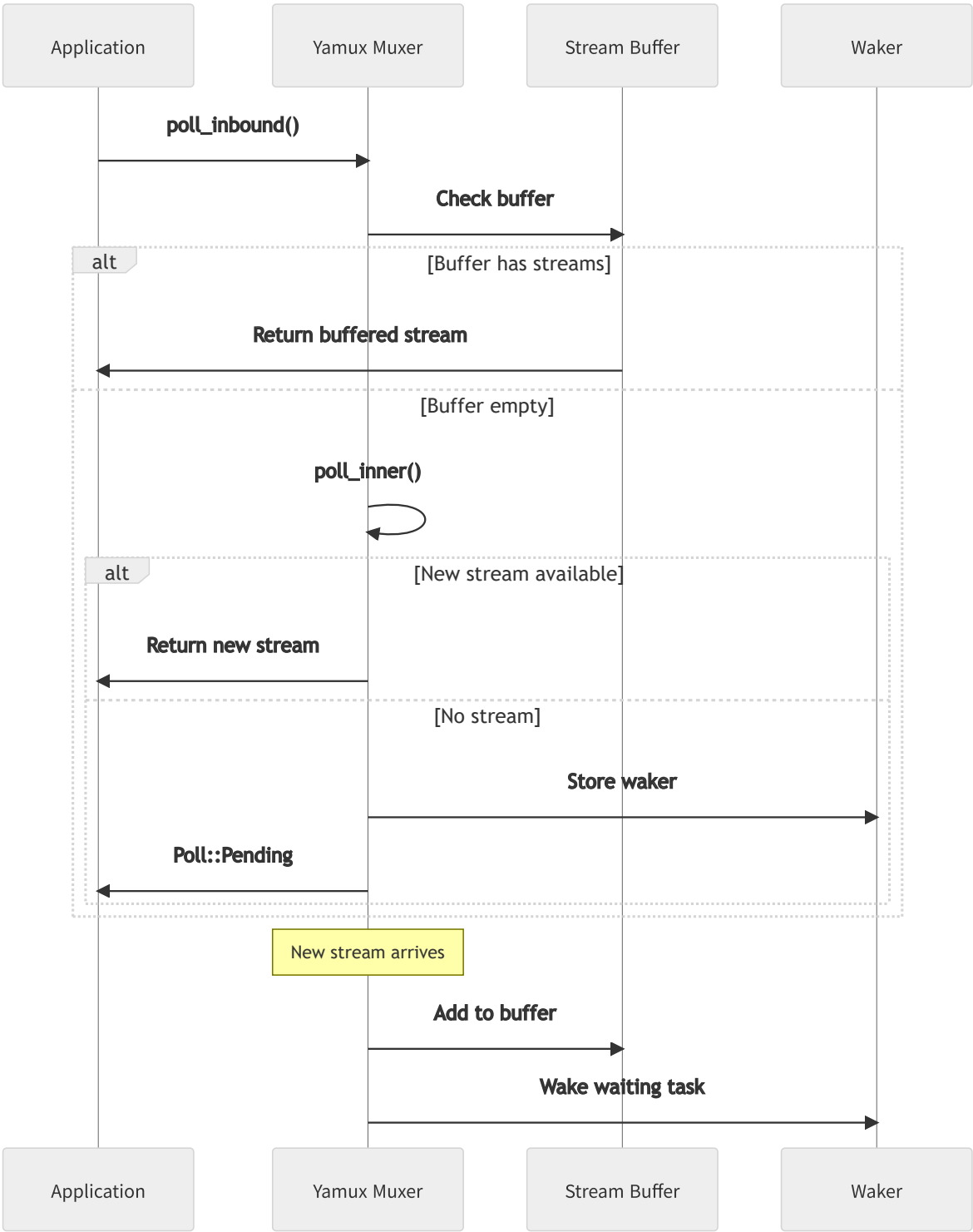


5. Multiplexing Layer 데이터 흐름

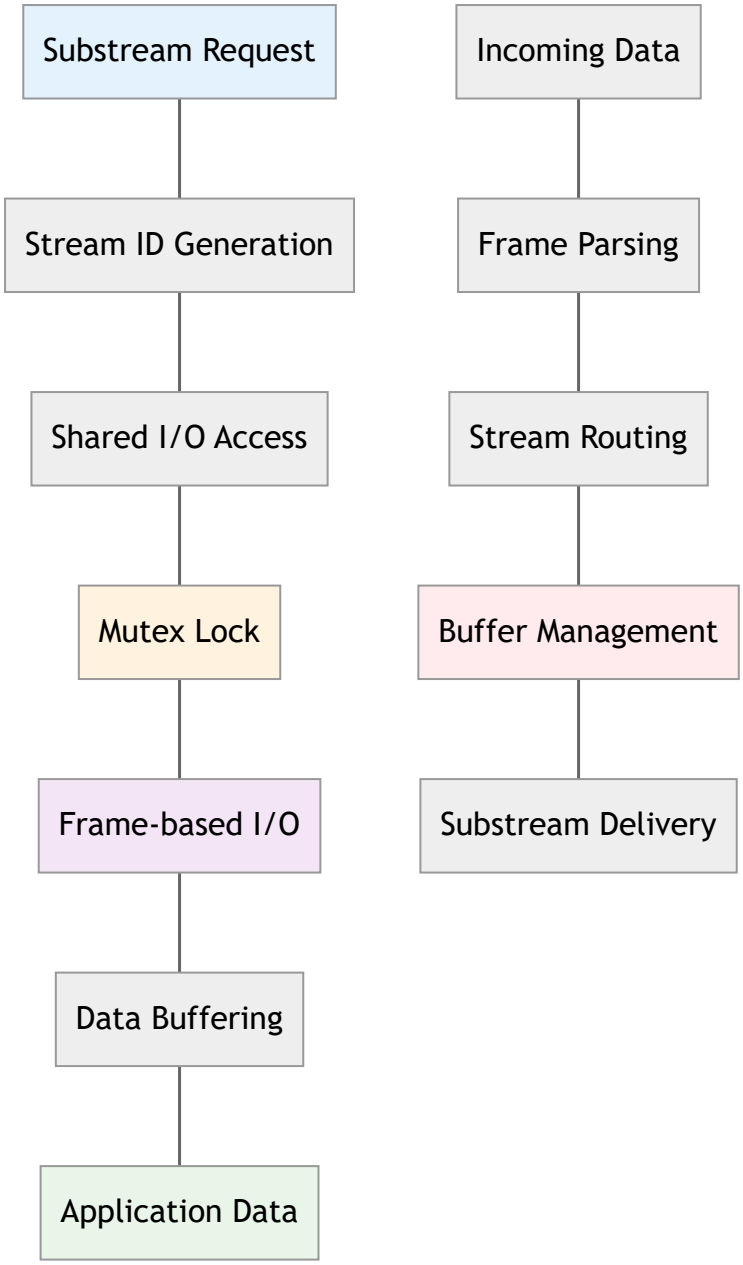
5.1 Yamux 스트림 멀티플렉싱



5.2 Yamux 백프레셔 메커니즘

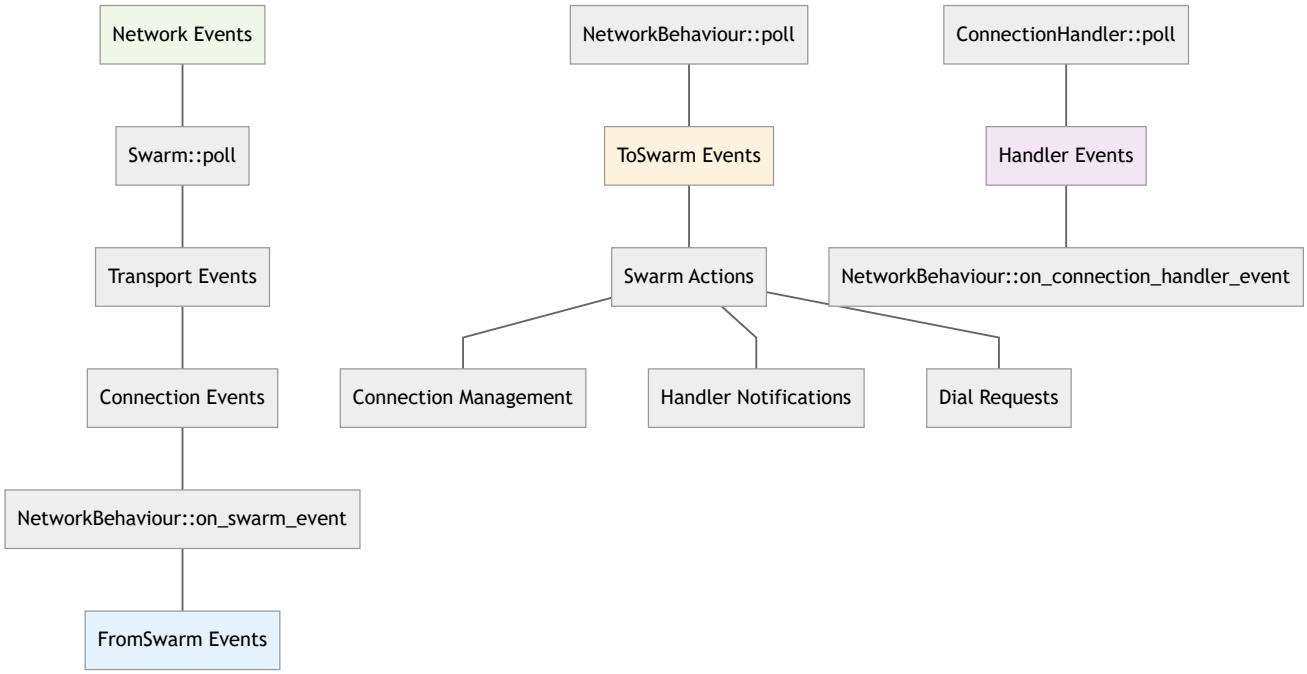


5.3 Mplex 스트림 처리

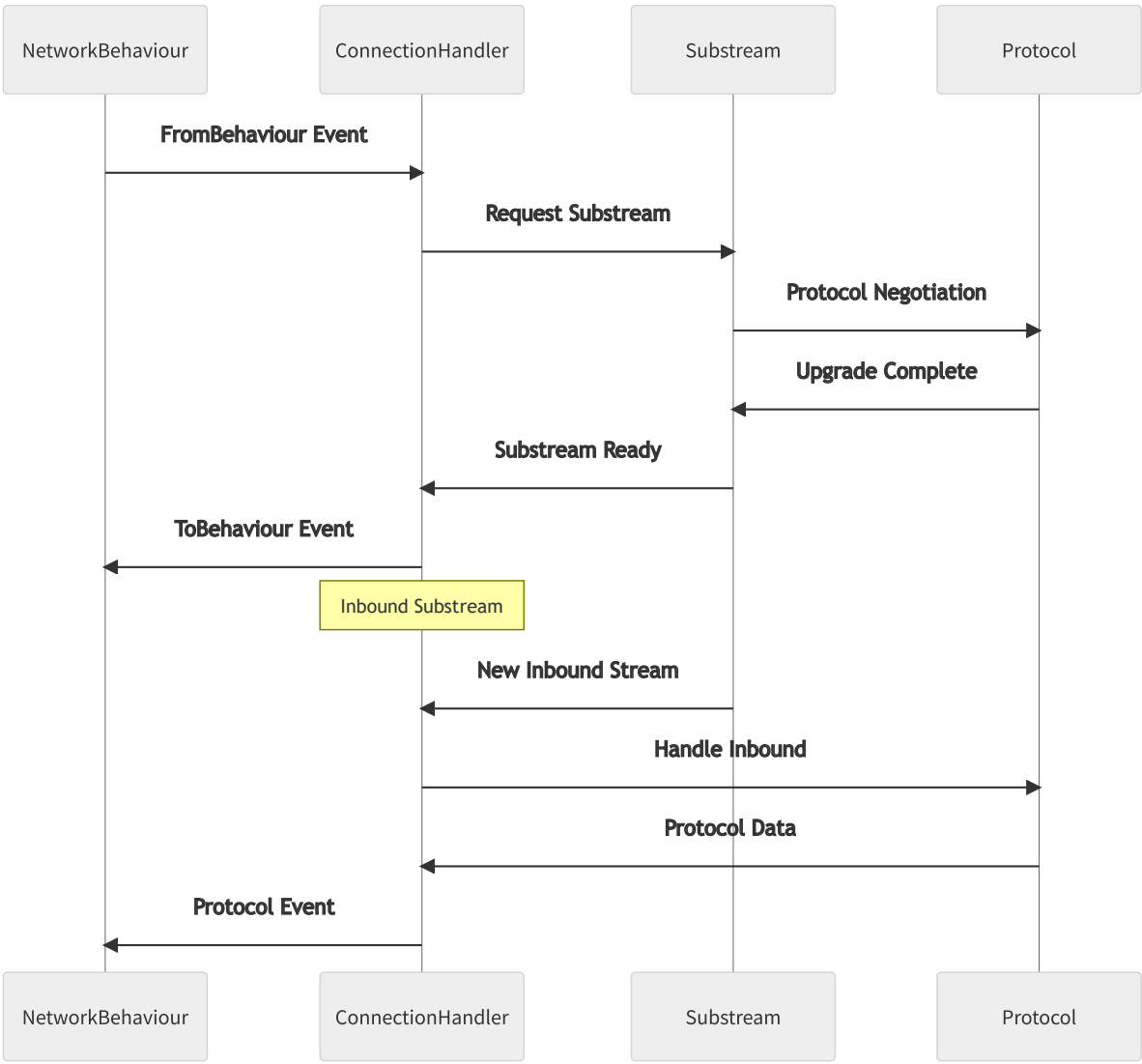


6. Swarm Layer 데이터 흐름

6.1 NetworkBehaviour 이벤트 흐름

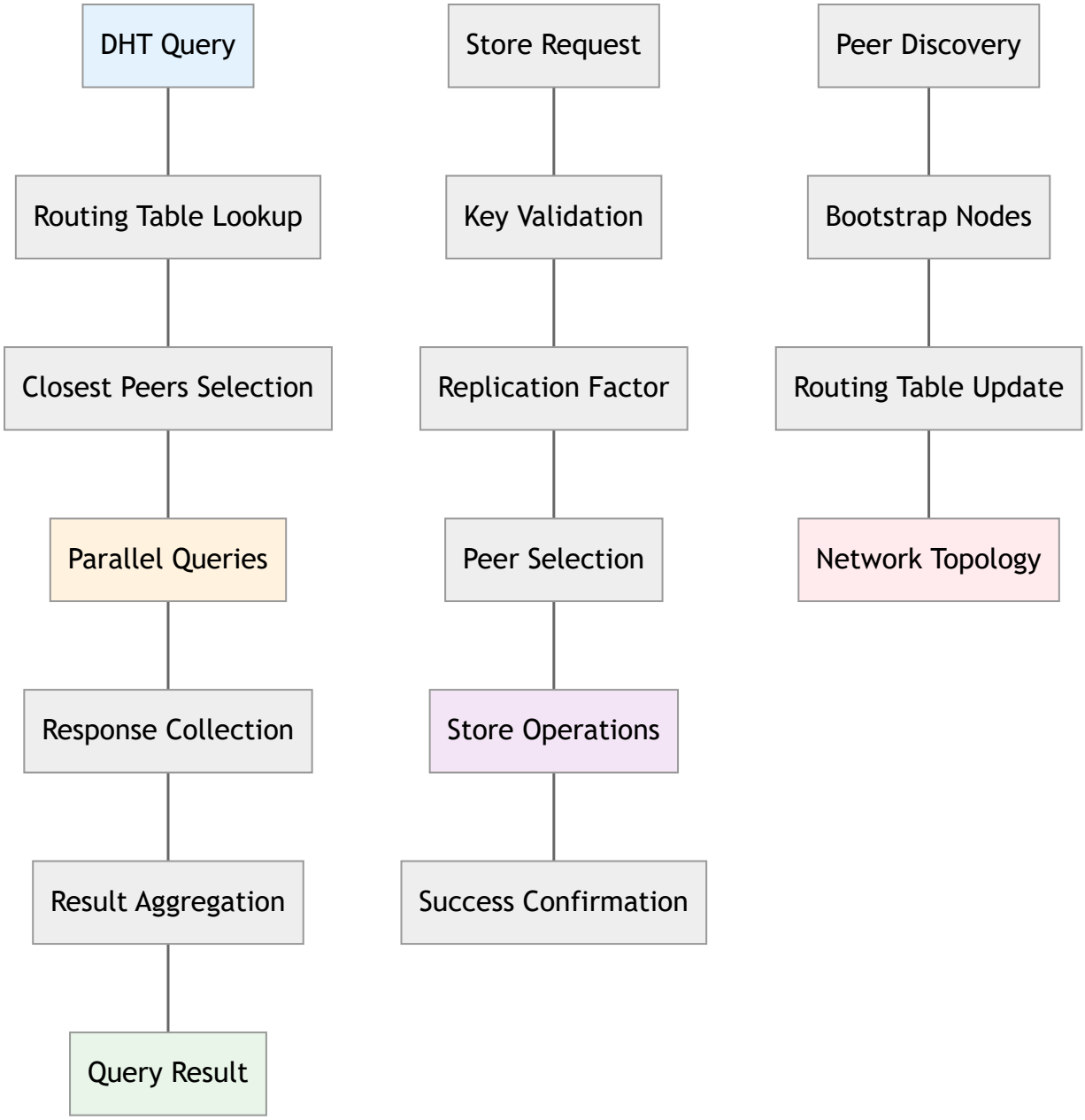


6.2 ConnectionHandler 데이터 흐름



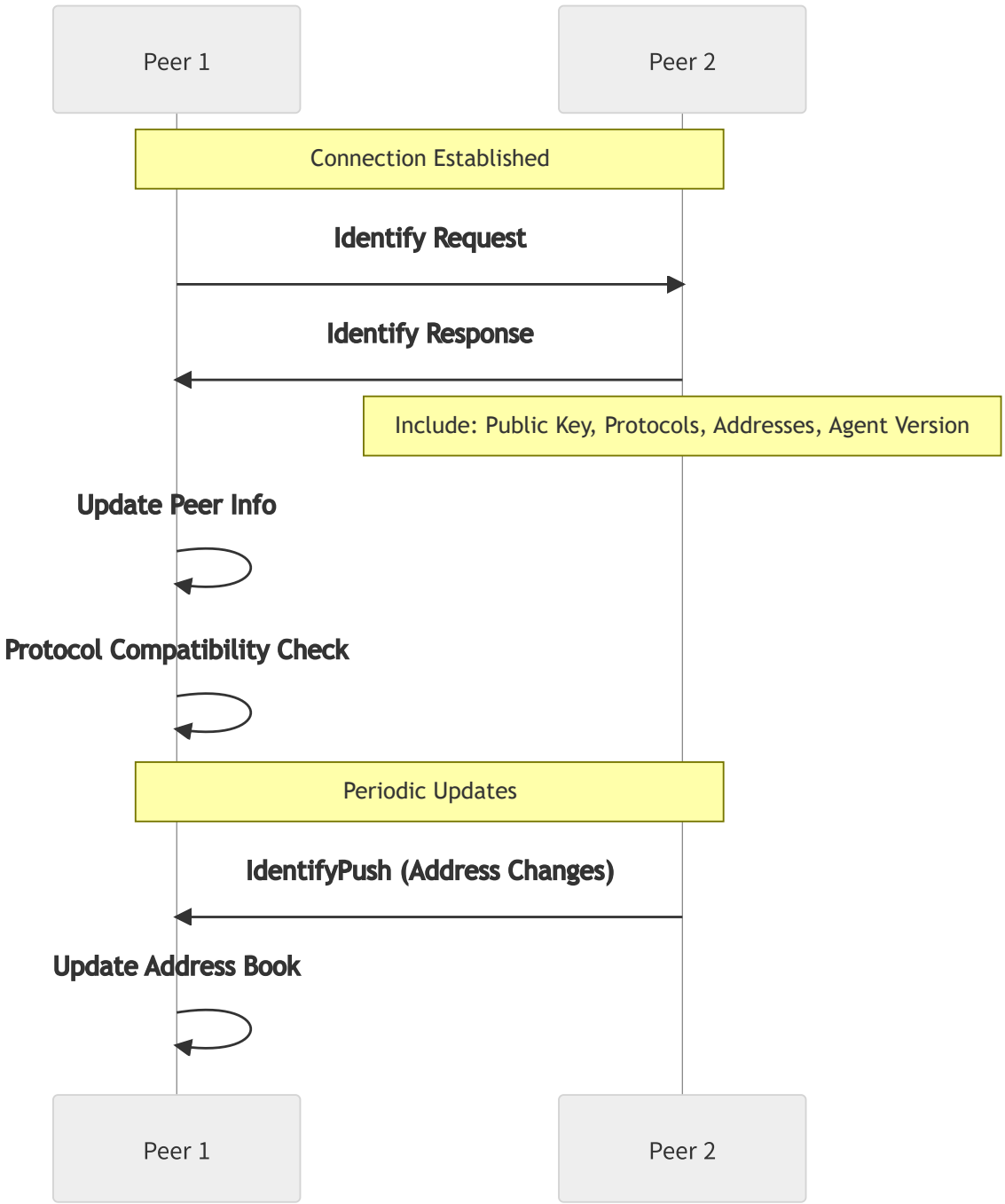
7. Protocol Layer 데이터 흐름

7.1 Kademlia DHT 데이터 흐름



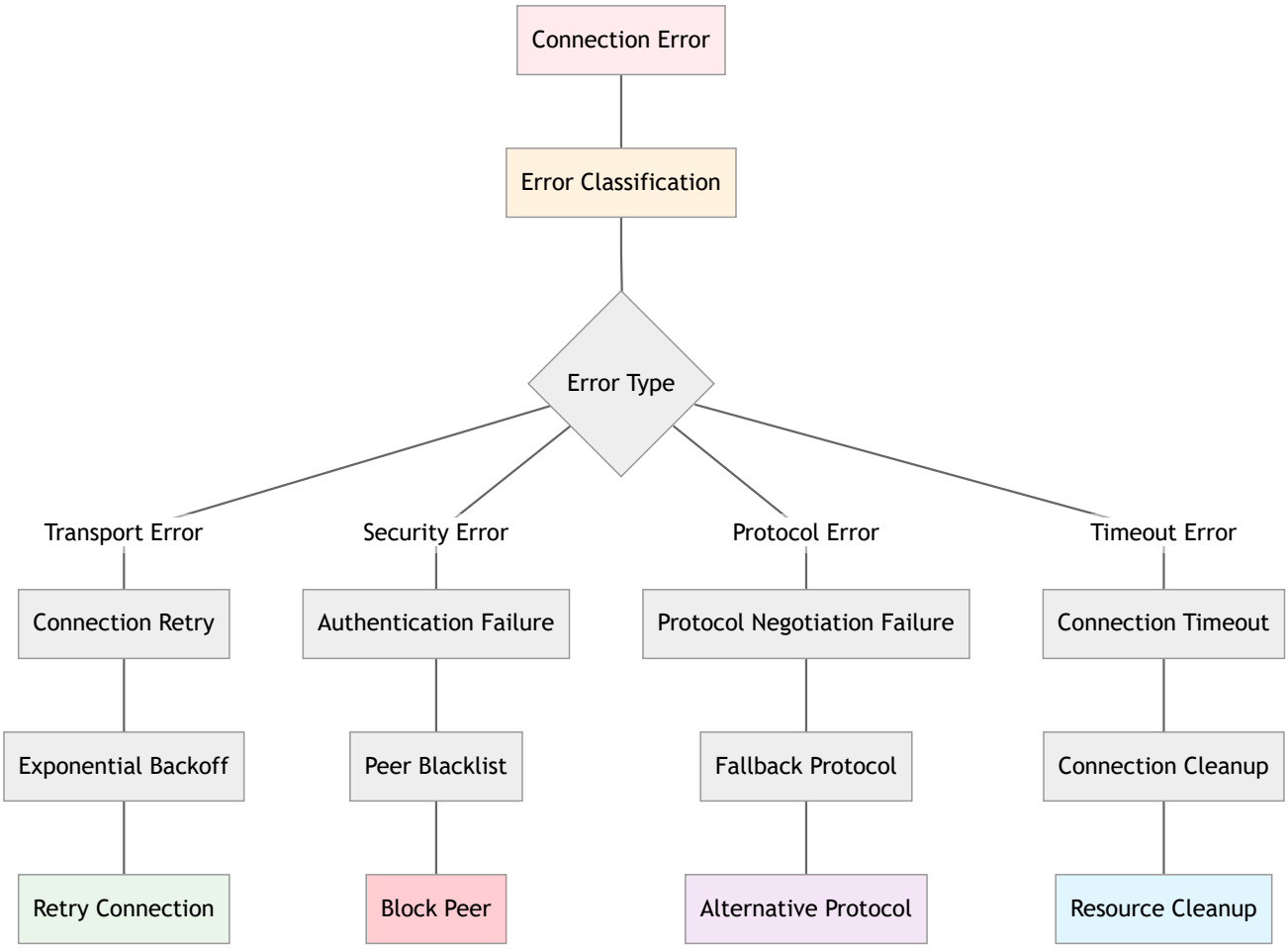
7.2 Gossipsub 메시지 전파

7.3 Identify Protocol 정보 교환

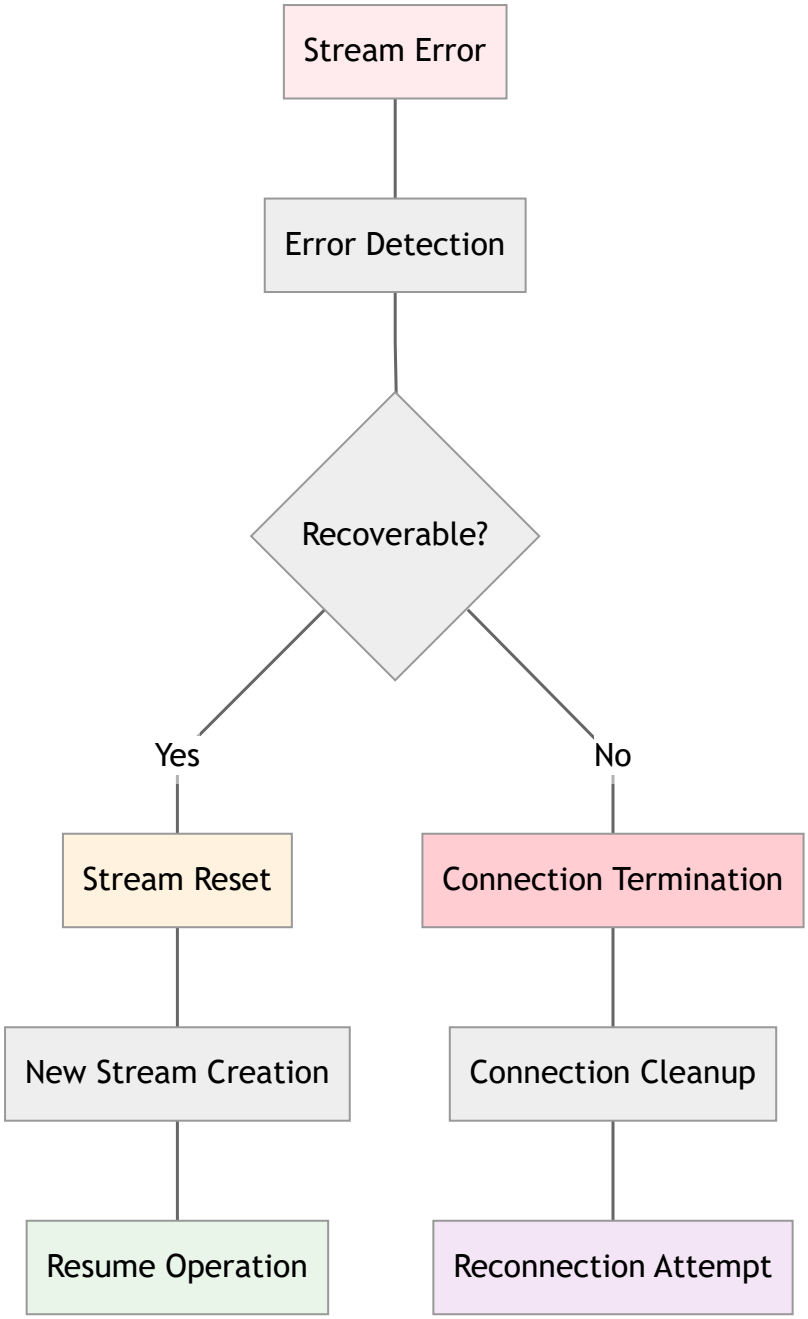


8. 오류 처리 데이터 흐름

8.1 연결 오류 처리

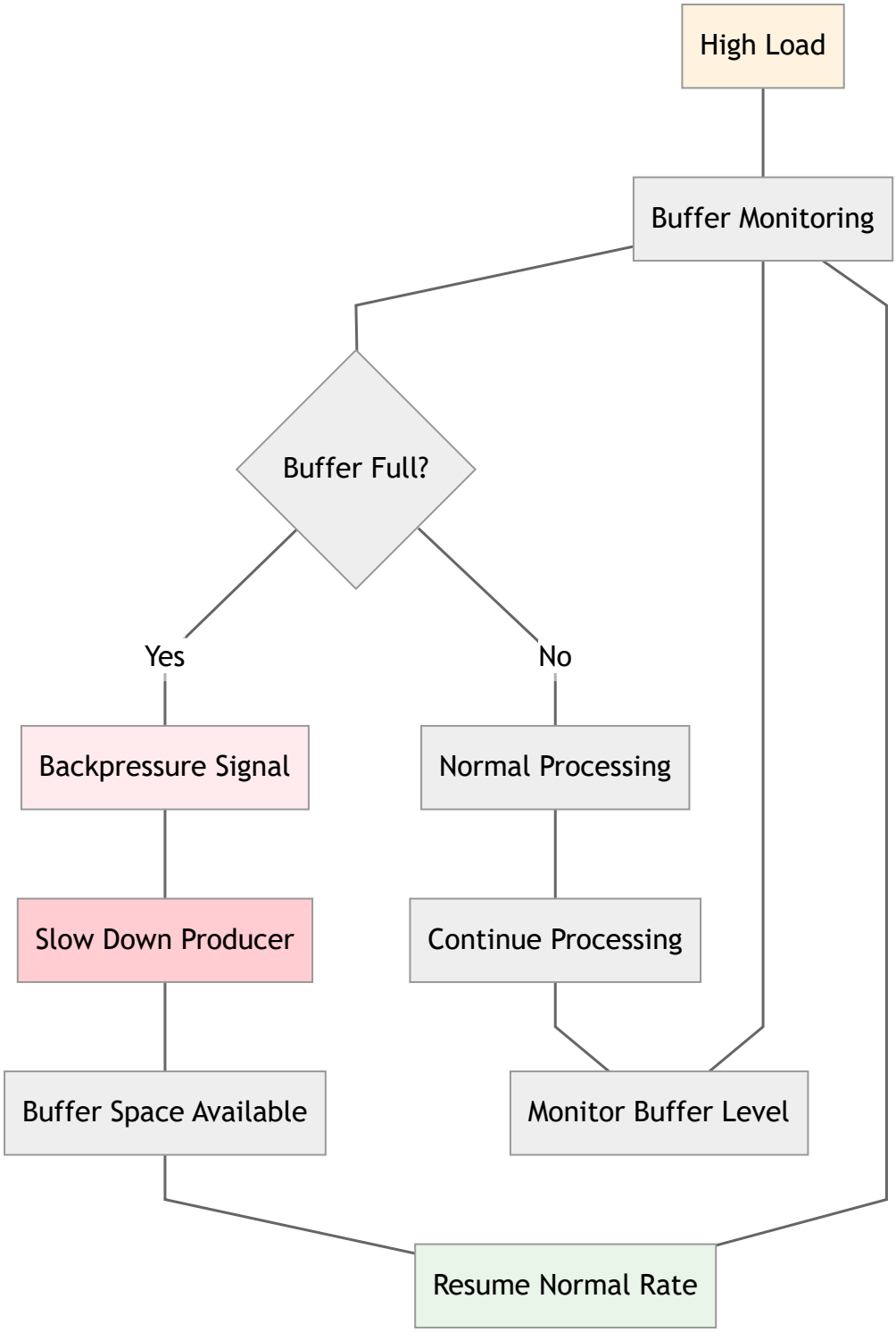

Connection Error
Error Classification
Error Type
Transport Error
Security Error
Protocol Error
Timeout Error
Connection Retry
Authentication Failure
Protocol Negotiation Failure
Connection Timeout
Exponential Backoff
Peer Blacklist
Fallback Protocol
Connection Cleanup
Retry Connection
Block Peer
Alternative Protocol
Resource Cleanup

8.2 스트림 오류 복구

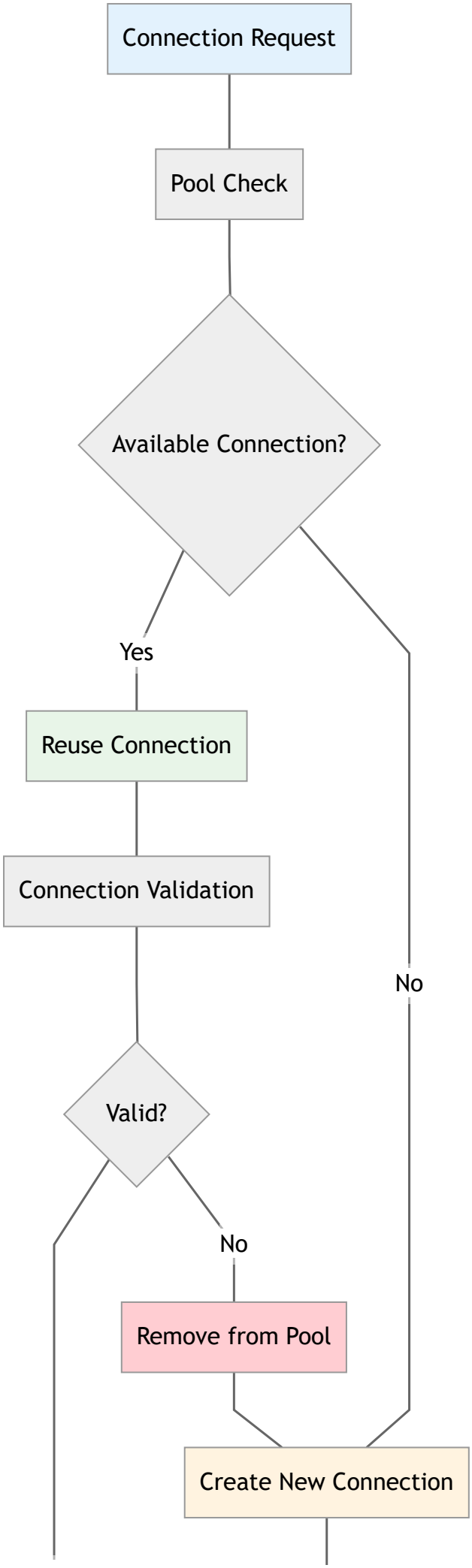


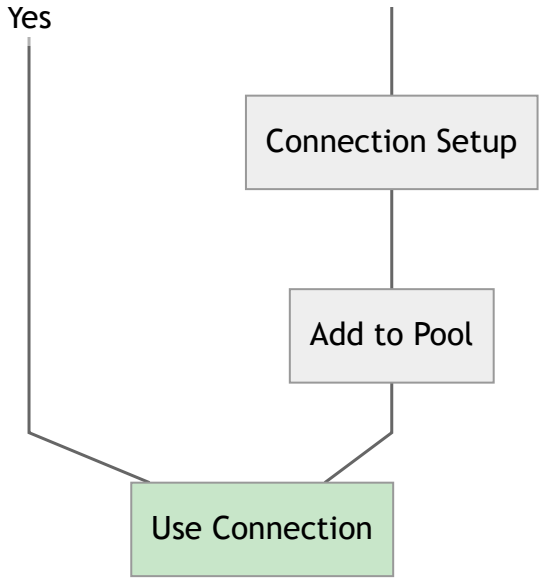
9. 성능 최적화 데이터 흐름

9.1 백프레셔 제어


High Load
Buffer Monitoring
Buffer Full?
Yes
No
Backpressure Signal
Normal Processing
Slow Down Producer
Continue Processing
Buffer Space Available
Monitor Buffer Level
Resume Normal Rate

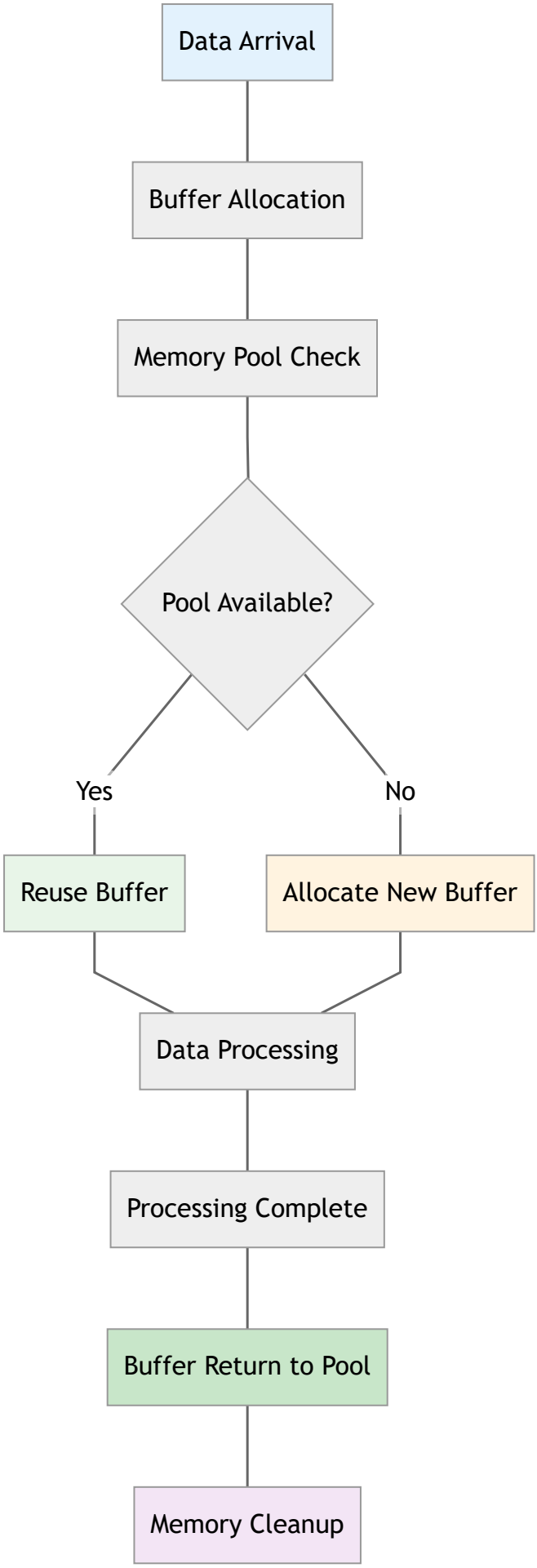
9.2 연결 풀링


Connection Request
Pool Check
Available Connection?
Yes
No
Reuse Connection
Connection Validation
Valid?
No
Remove from Pool
Create New Connection



10. 메모리 관리 데이터 흐름

10.1 버퍼 관리


Data Arrival
Buffer Allocation
Memory Pool Check
Pool Available?
Yes
No
Reuse Buffer
Allocate New Buffer
Data Processing
Processing Complete
Buffer Return to Pool
Memory Cleanup

결론

이러한 데이터 흐름 다이어그램들은 Rust libp2p의 복잡한 아키텍처를 시각적으로 이해할 수 있게 도와줍니다. 각 계층별 데이터 처리 방식과 구성 요소 간의 상호작용을 명확히 보여주어, 전체 시스템의 동작 원리를 파악하는 데 유용합니다.

주요 특징:

1. **계층적 구조**: 각 계층이 명확한 책임을 가지고 데이터를 처리
2. **비동기 처리**: 이벤트 기반 비동기 데이터 흐름
3. **오류 복구**: 강력한 오류 처리 및 복구 메커니즘
4. **성능 최적화**: 백프레셔, 연결 풀링, 메모리 관리 등
5. **확장성**: 모듈식 설계로 새로운 프로토콜 추가 용이

이러한 설계는 대규모 P2P 네트워크에서 안정적이고 효율적인 통신을 가능하게 합니다.